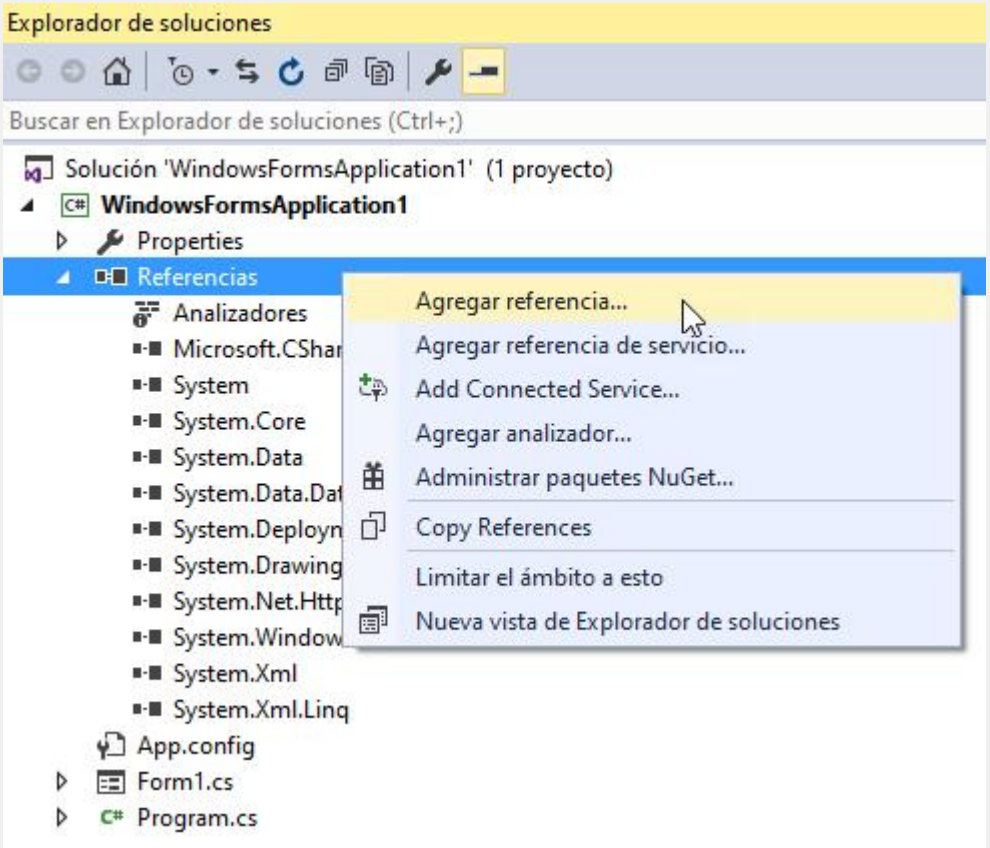
LN class (Library New)

The LN class has been conceived as a library of functions (methods) that support the C# programmer and that would save a large number of programming lines. On the other hand, it greatly reduces the loss of time produced by the "how to do it" in programming time. This document is basically oriented to the neophyte C# programmer and for this reason we try to be as explicit as possible in the document.

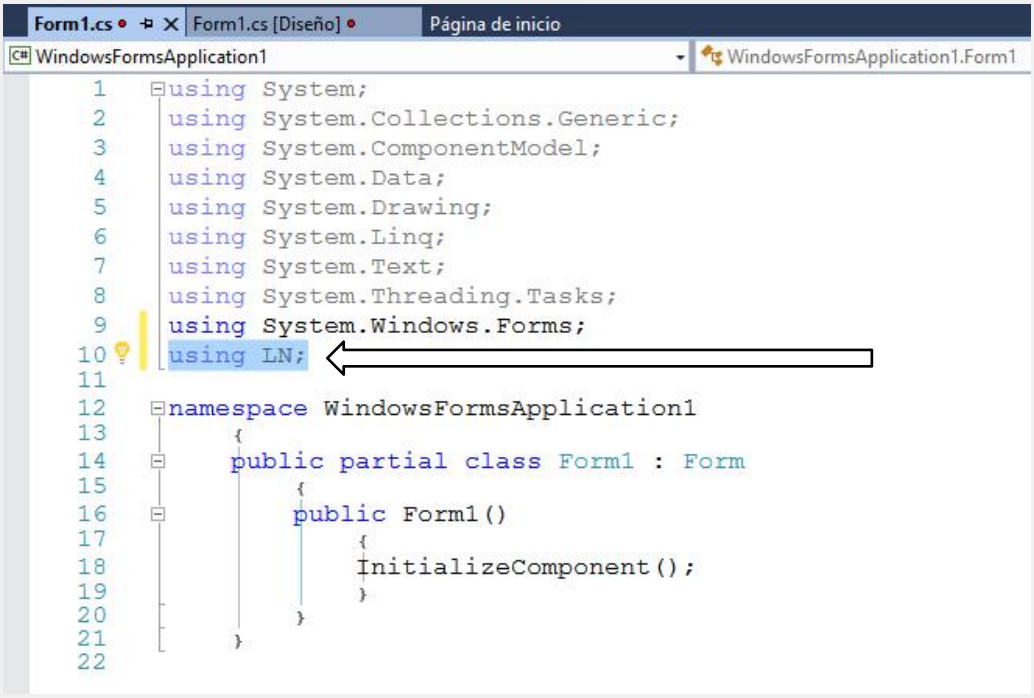
This class is represented by the dynamic library LN.DLL attached to this project.

In order to work with the methods of the LN class, the following steps must be followed:

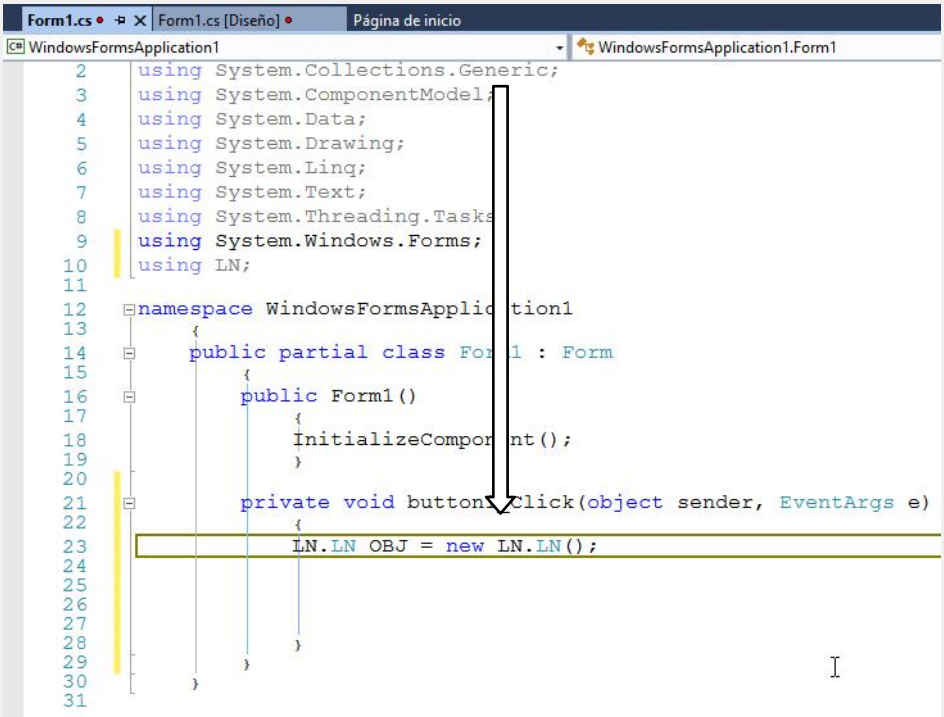
1) We start by adding a reference to said DLL, as follows:



2) Next we add the LN namespace using the corresponding using within the program code in the traditional way:



3) From this moment you can instantiate the object with the desired name (in our case, called OBJ):



4) Now when you write OBJ followed by a period, the complete list of all the methods of said class will quickly appear, and all you have to do is select the desired method.

The LN class is designed primarily for Winforms (.exe) type applications, but the document clearly specifies which methods can be used in
WEB applications and which in Winforms applications.

LN CLASS METHODS

METHODS OF ISSUING MESSAGES 6
public void MensINFO(string Message) 6
public void MensERROR(string Message) 6
public bool MessagesConfirm(string Message) 6
public void MsgTemp(string M, int Seg) 7
MailSender and SendMail 8
public void MailSender 9
public void SendMail(MailMessageMessageParameters) 11
WORK METHODS WITH DATES 11
public int CheckDateUS(string Date) 11
public int CheckDateESP(string Date) 12
public string NormalizeDateUS(string EntryDate) 13
public string NormalizeDateESP_Str(string EntryDate) 13
public string NormalizeDateISO_Str(string EntryDate) 14
public string ExtractHour_STR_HM(DateTime Date) 14
public string NormalizeHour_STR(stringEntryHour) 15
public string ExtractDateUS_Str(DateTime Date) 15
public string ExtractDateESP_Str(DateTime Date) 15
public string ConvertDateUStoESP_Str(string EntryDate) 15
public string ConvertDateESPtoUS_Str(string EntryDate) 16
public string ConvertDateISOtoUS_Str(string EntryDate) 16
public string ConvertDateISOtoESP_Str(string EntryDate) 17
public int AgeIntESP(string FNac) 17
public double AgeDecESP(string FNac) 18
public int AgeIntUS(string FNac) 19
public int AgeDecUS(string FNac) 20
public int AgeFromToESP(string From, string To) 20
public int AgeFromToUS(string From, string To) 21
public double AgeFromToDecESP(string From, string To) 22
public double AgeFromToDecUS(string From, string To) 23
public double DateDifD(DateTime FROM, DateTime TO) 23
public double DateDifH(DateTime FROM, DateTime TO) 24
public double DateDifM(DateTime FROM, DateTime TO) 25
public bool NormalizeDate(int A, int M, int D,ref int ANorm, ref int MNorm, ref int DNorm) 25
DATA VALIDATION METHODS 27
public bool SSNCheck(string SSN) 27
public bool NUM(string Number) 27
public bool NUMERIC(string Number) 28
public bool ZERO(string Number) 28
public bool ALFANUM(string String) 28
public bool ALF(string String) 29

public bool ALFEng(string String) 29
public bool Leap(string AAAAStr) 30
public bool BetweenInc(int N, int a, int b) 30
public bool BetweenExc(int N, int a, int b) 30
public bool BetweenIncDec(double N, double a, double b) 31
public bool BetweenExcDec(double N, double a, double b) 31
bool VINCheck(string VIN) 32
PROCESS EXECUTION METHODS 32
public int MSDOSWait(string command) 32
public int MSDOSNoWait(string command) 33
public void PAUSE() 33
public int ExecProcess(string command) 33
public int ExecProcessNoWait(string command) 34
STRING FUNCTION METHODS 35
public int FindStr(string String, string Substring) 35
public string FillString(string Char, int Ntimes) 36
public string DeleteChar(string String, string CHAR) 36
public string DeleteStr(string String, string PATTERN) 37
public void SplitString(ref string[ ] Result, string String, string[ ] delimiter) 38
GENERAL METHODS 39
public double Percent(double P, double N) 39
public string Trunc(double Origin, int Integers, int Decimals) 40
public void Upper(ref string String) 40
public void Lower(ref string String) 41
public void ShowRPT(string PathReport) 41
FileDiskOpenCreate method 42
FileDiskOpenAppend method 43
FileDiskOpenRead method 44
FileDiskWrite method 45
FileDiskRead method 46
FileDiskCloseWriter method 47
FileDiskCloseReader method 48
GetDirectoryFilesDialog method 49
GetDirectoryFilesPath method 50
GetDirectoryFoldersDialog method 51
GetDirectoryFoldersPath method 52
CreateFileParameter method 53
CreateFileParameterArray method 55
MethodStorageSize 56

METHODS OF ISSUING MESSAGES

public void MensINFO(string Message)

Usage: Winforms App
Description:
Issues an informational message with the text that appears in the input parameter.

Input Parameters:
string Message

Return:
Does not have

public void MensERROR(string Message)

Usage: Winforms App
Description:
Issues an error message with the text that appears
in the input parameter.

Input Parameters:
Message

Return:
Does not have

public bool MessagesConfirm(string Message)

Usage: Winforms App
Description:
Issues a confirmation request message whose text is
in the input parameter.

Input Parameters:
string Message

Return:
True –Accepted
False -Denied

public void MsgTemp(string M, int Seg)

Usage: Winforms App
PARAMETERS:
string M
int Sec

RETURN:
DOES NOT HAVE

DESCRIPTION:

The MsgTemp method displays a temporary message so that when
When the elapsed time expires, the message disappears.and continues
the execution of the program.

The input parameter M of type string contains the message to
display, while the second parameter Seg specifies the
number of seconds the message will remain on the screen.

The maximum message length is 60 characters, otherwise
an error message occurs.

Example:

```
LN.LN Object = new LN.LN();

...

Object.MsgTemp("Please wait a moment...", 3);
```

*This message lasts only 3 seconds before disappearing

MailSender and SendMail

Usage: Winforms App & WEB App

The MailSender and SendMail methods that will be explained below are designed to automate the sending of an e-mail to one or more recipients from a program written in C#, however, it is necessary to clarify the GMAIL warning in this regard, which shown below:

==========

You need to allow access to non-secure apps from Gmail settings as described in the Gmail documentation.

If you do not enable it, you will receive a security alert from Google indicating that an application other than Gmail has accessed your email and the application will show you this message.

The SMTP server requires a secure connection or the client was not authenticated. The server response was: 5.7.0 Authentication Required.

Warning*The .NET team has marked the class SmtpClient as obsolete. Instead they recommend MailKit*

Gmail Settings

The necessary configuration for the SMTP client according to the Gmail documentation is the next:

Parameter	Worth
Outgoing mail server (SMTP)	smtp.gmail.com
Requires TLS	Yes (if available)
Requires authentication	Yeah
Port for TLS/STARTTLS	587
Full name or display name	Your name
Account name, username, or email address	Your full email address
Password	Your Gmail password

To use these methods in other providers such as Yahoo, Hotmail or any other, information must be obtained from them, if required.

public void MailSender

Usage: Winforms App & WEB App
MailSenderIt is a declarative method that establishes email parameters in a declarative way.

PARAMETERS:
(string SMTP,
int PORT,
string Login,
string Password,
string SUBJECT,
string FROM,
string[] TO,
string ATTACH,
string[] MessageBody)

DESCRIPTION:

The MailSender method is the main messaging method for sending mail from the program and it establishes all the necessary parameters for the e-mail to be transmitted correctly.

The parameters are as follows:

IMPORTANT NOTE FOR GMAIL EMAIL(Extracted from GOOGLE):

Gmail's SMTP service is offered with sending limitations to prevent spammers from using its service to send junk emails.

The limits set 100 recipients at a time, and 500 messages per day: if you exceed this restriction, Google will block your account.

In addition, its SMTP service uses shared IPs that do not guarantee delivery, since these are also used by spammers who seriously damage the reputation of these addresses, increasing the chances that your legitimate email will be filtered by antispam systems by mistake.

For this reason, if you are sending an email campaign or need to send unlimited emails, the best alternative is to opt for a professional SMTP server contracted from the service provider and for a certain price you will be able to send without restrictions to the users you want. .

INPUT PARAMETERS:

SMTP string,

Here you declare the address of the sending server (SMTP) that corresponds to the messaging client you want to use.

For example:

"smtp.gmail.com"	for GMAIL email
"smtp.mail.yahoo.com"	for Yahoo mail
"smtp.live.com"	for HOTMAIL mail

---

int PORT,

Gmail ==>	**Port 587 (25 or 465 may present problems)**
Yahoo ==>	**Port 587 (25 or 465 may present problems)**
Hotmail ==>	**Port 25 or 465**

Important note:

It is worth clarifying that large internet service providers such as GMAIL, HOTMAIL, etc., usually establish a certain level of protection against the issuance of messages by program, for example in the case of GMAIL, Google always warns the recipient that the message received It is implicitly considered SPAM in such a way as to force the sender to make the service that is intended to be sent official.
scheduled messages to your clients or recipients.

string Login, (user login)
string Password,(Login Password). Gmail does not accept the normal account password: you must turn on 2-Step Verification and create an "app password" (see support.google.com/accounts/answer/185833)
string SUBJECT, (Mail subject title)
string FROM, (Sender email address)
string[] TO, (Recipient's email address)
string ATTACH, (Files attached to transmit)
string[] MessageBody)(Message body)
RETURN: DOES NOT HAVE

public void SendMail(MailMessage MessageParameters)

Usage: Winforms App & WEB App
Sends the email based on the parameters defined by the MailSender method

INPUT PARAMETERS:
MailMessage MessageParameters

DESCRIPTION:
The MessageParameters object contains a structure of type **MailMessage** contains all the information previously provided by the MailSender method.

WORK METHODS WITH DATES

Usage: Winforms App & WEB App

public int CheckDateUS(string Date)

DESCRIPTION:

This method checks a date in US format ("mm/dd/yyyy").

INPUT PARAMETERS:

string Date (in "mm/dd/yyyy" format)

RETURN int type:

1 - OK
2 - "dd" is not numeric
3 - "dd" out of range
4 - "mm" is not numeric
5 - "mm" out of range
6 - "yyyy" is not numeric
7 - First digit of “yyyy” is not 1 or 2
8 - Second digit of "yyyy" is not 9 or 0

9 - Does not have slash in position 3 or 6
10 - Wrong length
11- Month of February with more than 29 days in
leap-year
12- Month of February with more than 28 days in
non-leap year
13- The months of April, June, September,
or November
They should not be more than 30 days old

public int CheckDateESP(string Date)

DESCRIPTION:

This method performs the verification of a date with ESN format "dd/mm/yyyy"

INPUT PARAMETERS:

string Date ("dd/mm/yyyy")

RETURN:

1 - OK
2 - "dd" is not numeric
3 - "dd" out of range
4 - "mm" is not numeric
5 - "mm" out of range
6 - "yyyy" is not numeric
7 - First digit of “yyyy” is not 1 or 2
8 - Second digit of "yyyy" is not 9 or 0
9 - Does not have slash in position 3 or 6
10 - Wrong length
11- Month of February with more than 29 days in
leap-year
12- Month of February with more than 28 days in
non-leap year
13- The months of April, June, September,
o November must not be more than 30 days

public string NormalizeDateUS(string EntryDate)

DESCRIPTION:

This method normalizes the entry date that appears in the variable
DateInput in US format (m/d/yyyy) not normalized to normalized US format ("mm/dd/yyyy") returning the latter as a string.

The date is considered normalized when the year is expressed with four digits, the month with two digits and the day with two digits.

Example:

If the input is: "2/1/1980", then returns: "02/01/1980"

INPUT PARAMETERS:

string EntryDate

RETURN:

string **(WITH THE NORMALIZED DATE)**

If the entry date is incorrect, the string "0" is returned.
It is worth clarifying that the year of entry must necessarily have 4 digits.

public string NormalizeDateESP_Str(string EntryDate)

DESCRIPTION:
This method is similar to NormalizeDateUS but assuming that the input date is in the format "dd/mm/yyyy"

INPUT PARAMETERS:
string EntryDate

RETURN:
string **(WITH THE NORMALIZED DATE)**
If the entry date is incorrect, the string "0" is returned.
It is worth clarifying that the year of entry must necessarily have 4 digits.

public string NormalizeDateISO_Str(string EntryDate)

DESCRIPTION:
This method normalizes the entry date that appears in the variable **Entry date**in ISO format (yyyy-md) not normalized to ISO format normalized (yyyy-mm-dd) returning to the latter.

The date is considered normalized when the year is expressed with four digits, the month with two digits and the day with two digits.

INPUT PARAMETERS:
string EntryDate

RETURN:
string

Example:

Entrance: **1980-1-1**
Released: 1980-01-01

public string ExtractHour_STR_HM(DateTime Date)

DESCRIPTION:
This method extracts the date from the DateTime, returning it normalized in string format.

INPUT PARAMETERS:
DateTime Date

RETURN:
string ("0" if there was an error)

public string NormalizeHour_STR(stringEntryHour)

Description:
This method normalizes the entry time that appears in the variable HoraEntrada ("h:m:s") in normalized format ("hh:mm:ss").

INPUT PARAMETERS:
string EntryTime

Return:
string - (“0” if there was an error)
(“hh:mm:ss”) – Normalized if everything OK

public string ExtractDateUS_Str(DateTime Date)

DESCRIPTION:
This method extracts the date from the input parameter in US format (m/d/yyyy), returning it normalized (mm/dd/yyyy) in string format.

INPUT PARAMETERS:
DateTime Date

RETURN:
string

public string ExtractDateESP_Str(DateTime Date)

DESCRIPTION:
This method extracts the date from the DateTime with ESP format (dd/mm/yyyy) returning it normalized in string format.

INPUT PARAMETERS:
DateTime Date

RETURN:
string

public string ConvertDateUStoESP_Str(string EntryDate)

DESCRIPTION:

This method converts the entry date that appears in the variable EntryDate into US format ("mm/dd/yyyy") to ESP format ("dd/mm/yyyy") returning to the latter in string format.

Example:
Entry: "1/5/1980" (January 5, 1980 in US format)
Release: "01/05/1980" (January 5, 1980 in ESP format)

INPUT PARAMETERS:
string EntryDate

RETURN:
string (with the date in "dd/mm/yyyy" format)

If the entry date is incorrect, the string "0" is returned.
It is worth clarifying that the year of entry must necessarily have 4 digits.

public string ConvertDateESPtoUS_Str(string EntryDate)

DESCRIPTION:
This method converts the entry date that appears in the variable EntryDate into ESP format ("dd/mm/yyyy"), to US format ("mm/dd/yyyy") returning to the latter in string format.

Example:
Entry: "5/1/1980" (January 5, 1980 in ESP format)
Release: "05/01/1980" (January 5, 1980 in US format)

INPUT PARAMETERS:
string EntryDate

RETURN:
string (with the date in "dd/mm/yyyy" format)

If the entry date is incorrect, the string "0" is returned.
It is worth clarifying that the year of entry must necessarily have 4 digits.

public string ConvertDateISOtoUS_Str(string EntryDate)

DESCRIPTION:
This method converts the entry date that appears in the variable EntryDate into
ISO format ("yyyy-mm-dd") to US format ("mm/dd/yyyy") returning to the latter with string format.

INPUT PARAMETERS:
string EntryDate

RETURN:
string (with the date in "mm/dd/yyyy" format)

public string ConvertDateISOtoESP_Str(string EntryDate)

DESCRIPTION:
This method converts the entry date that appears in the variable EntryDate into
ISO format (yyyy-mm-dd) to Spanish format (dd/mm/yyyy), returning the latter as a string.

INPUT PARAMETERS:
string EntryDate

RETURN:
string

public int AgeIntESP(string FNac)

DESCRIPTION:
This method returns a person's age to date as an integer, with their Date of Birth as input argument in string format and ESN regionalization (Spanish="dd/mm/yyyy").

Please note that the entry date has to be normalized, that is, two digits for the day, two digits for the month and 4 digits for the year.

Example of use:
LN.LN OBJ = new LN.LN(); // Instantiate the LN type OBJ object
int ALPHA = OBJ.ESPStrAge(@"06/28/1948");
leaving ALPHA = 70 (Assuming that today is September 22, 2020)

IMPORTANT NOTE:

At the beginning of the execution of this method it is ensured that the input format is Spanish from Mexico, which matches many date formats of the type "dd/mm/yyyy" in Latin America. If you want the possible values for the CultureInfo argument you can visit the site on the Internet: https://msdn.microsoft.com/en-us/library/cc233982.aspx

For the United States, "en-US" is used where the date format is "mm/dd/yyyy".

INPUT PARAMETERS:
string FNac

RETURN:
-1 ==> If there was a format error in the entry date
age ==> If everything was OK

public double AgeDecESP(string FNac)

DESCRIPTION:
This method returns the age of a person to date as a decimal number, having as input argument their Date of Birth in string format and Spanish regionalization = "dd/mm/yyyy". Unlike the AgeESP method, here the exact date is given in decimal form.

Please note that the entry date has to be normalized, that is, two digits for the day, two digits for the month and 4 digits for the year.

Example of use:

```
using LN;
...
LN.LN OBJ = new LN.LN(); // Instantiate the LN type OBJ object
double ALPHA = OBJ.ESPDecAge(@"06/28/1948");
```

IMPORTANT NOTE:
At the beginning of the execution of this method it is ensured that the input format is Spanish from Mexico, which matches many date formats of the type "dd/mm/yyyy" in Latin America. If you want the possible values for the CultureInfo argument you can visit the site on the Internet:

https://msdn.microsoft.com/en-us/library/cc233982.aspx

For the United States, "en-US" is used where the date format is "mm/dd/yyyy".

INPUT PARAMETERS:
string FNac

RETURN:
-1 ==> If there was a format error in the entry date
age ==> If everything was OK

public int AgeIntUS(string FNac)

DESCRIPTION:
This method returns a person's age to date as an integer, with their Date of Birth as input argument in string format and US regionalization ("mm/dd/yyyy"). Please note that the entry date has to be normalized, that is, two digits for the month, two digits for the day and 4 digits for the year.

Example of use:

```
LN.LN OBJ = new LN.LN(); // Instantiate the LN type OBJ object
int ALPHA = OBJ.IntUSAge(@"06/28/1948");
```

leaving ALPHA = 70 (Assuming today is December 16, 2019)

IMPORTANT NOTE:

At the beginning of the execution of this method, it is ensured that the input format is United States English.

If you want the possible values for the CultureInfo argument you can visit the site on the Internet: https://msdn.microsoft.com/en-us/library/cc233982.aspx

For the United States, "en-US" is used where the date format is "mm/dd/yyyy".

INPUT PARAMETERS:
string FNac

RETURN:
-1 ==> If there was a format error in the entry date
age ==> If everything was OK

public int AgeDecUS(string FNac)

DESCRIPTION:
This method returns a person's age to date as a decimal number, with their Date of Birth as input argument in string format and US regionalization ("mm/dd/yyyy"). Please note that the entry date has to be normalized, that is, two digits for the month, two digits for the day and 4 digits for the year.

Example of use:
LN.LN OBJ = new LN.LN(); // Instantiate the LN type OBJ object
int ALPHA = OBJ.DecAge(@"06/28/1948");

leaving ALPHA in decimal number format.

IMPORTANT NOTE:

At the beginning of the execution of this method, it is ensured that the input format is US English. If you want the possible values for the CultureInfo argument you can visit Internet site:

https://msdn.microsoft.com/en-us/library/cc233982.aspx

For the United States, "en-US" is used where the date format is "mm/dd/yyyy".

INPUT PARAMETERS:
string FNac

RETURN:
-1 ==> If there was a format error in the entry date
age ==> If everything was OK

public int AgeFromToESP(string From, string To)

DESCRIPTION:
This method returns the difference To - From in years as an integer, taking as input arguments the From and To date in string format.

It is a requirement that the date (Until be after From, if this is not met or if it exists some other error in the input parameters then: -1 is returned

In this method it is assumed that the entry dates are in Spanish regionalization (dd/mm/yyyy).

INPUT PARAMETERS:
string From
string Until

RETURN:
int

Example of use:
using LN;
...
LN.LN OBJ = new LN.LN();
. . .
int ALPHA = OBJ.AgeFromToESP(@"06/28/1948", @"12/16/2019");
gives a result in ALPHA as an integer.

public int AgeFromToUS(string From, string To)

DESCRIPTION:
This method Returns the age of a person as an integer with as input arguments their Date of Birth in string format and the Until date in string format.

It is a requirement that the date (Until) be later than From, if this is not met or if there is any other error in the input parameters then: -1 is returned

In this method it is assumed that the entry dates are in USA regionalization (mm/dd/yyyy).

INPUT PARAMETERS:
string From
string Until

RETURN:
int

Example of use:

```
using LN;
...
LN.LN OBJ = new LN.LN();
. . .
int ALPHA = OBJ.AgeFromToUS(@"06/28/1948", @"12/16/2019");

results:
ALPHA = 70
```

public double AgeFromToDecESP(string From, string To)

DESCRIPTION:
This method returns the age of a person as a decimal having as input arguments their Date of Birth in string format and the Until date in string format.

It is a requirement that the date (Until) be later than From, if this is not met or if there is any other error in the input parameters then: -1 is returned

In this method it is assumed that the entry dates are in Spain regionalization (¨dd/mm/yyyy¨).

INPUT PARAMETERS:
string From
string Until

RETURN:
double

Example of use:

```
using LN;
...
LN.LN OBJ = new LN.LN();
. . .
int ALPHA = OBJ.AgeFromToUS(@"06/28/1948", @"12/16/2019");
results:
ALPHA = 70.63
```

public double AgeFromToDecUS(string From, string To)

DESCRIPTION:
This method returns the age of a person as a decimal having as input arguments From in string format and To in string format.

It is a requirement that the To date be later than From, if this is not met or if there is any other error in the input parameters then: -1 is returned

In this method it is assumed that the entry dates are in USA regionalization (¨mm/dd/yyyy¨).

INPUT PARAMETERS:
string From
string Until

RETURN:
int

Example of use:

```
using LN;
...
LN.LN OBJ = new LN.LN();
. . .
int ALPHA = OBJ.AgeFromToUS(@"06/28/1948", @"12/16/2019");

results:
ALPHA = 70.63
```

public double DateDifD(DateTime FROM, DateTime TO)

DESCRIPTION:

Method that there is the difference in days between the FROM and TO dates
(TO - FROM).

Input Parameters:
DateTime FROM
DateTime TO

Return:
-1 **UNTIL**is not later than SINCE.
-1 Input parameters cannot be null.

A decimal number greater than or equal to 0 if everything was correct, where this number represents the difference in days from TO - FROM.

NOTE:
When you want to enter a specific date for FROM or TO, you can do it as shown below:

DateTime SINCE = new DateTime(2018, 08, 25);

And then the date would be FROM under the C# DateTime format.

public double DateDifH(DateTime FROM, DateTime TO)

DESCRIPTION:
Method that there is the difference in hours between the FROM and TO dates
(TO - FROM).

Input Parameters:
DateTime FROM
DateTime TO

Return:
-1 **UNTIL**is not later than SINCE.
-1 Input parameters cannot be null.

A decimal number greater than or equal to 0 if everything was correct, where this number represents the difference in hours from TO - FROM.

NOTE:
When you want to enter a specific date for FROM or TO, you can do it as shown below:

DateTime SINCE = new DateTime(2018, 08, 25);

And then the date would be FROM under the C# DateTime format.

public double DateDifM(DateTime FROM, DateTime TO)

DESCRIPTION:

Method that there is the difference in months between the FROM and TO dates
(TO - FROM).

Important Clarification:
Since the number of days in the months can vary in the case of February depending on whether the year is a leap year or not, we make a special consideration of assuming all months have 30 days.

Input Parameters:
DateTime FROM
DateTime TO

Return:
-1 **UNTIL**is not later than SINCE.
-1 Input parameters cannot be null.

A decimal number greater than or equal to 0 if everything was correct, where this number represents the difference in months from TO - FROM.

NOTE:
When you want to enter a specific date for FROM or TO, you can do it as shown below:

DateTime SINCE = new DateTime(2018, 08, 25);

And then the date would be FROM under the C# DateTime format.

public bool NormalizeDate(int A, int M, int D,ref int ANorm, ref int MNorm, ref int DNorm)

Description:
This function performs the normalization of the quantities given in the first three parameters in such a way that if the number of months in a year is greater than 12 or if the number of days is greater than 30, the necessary adjustment is made, leaving the result in the output variables ANorm, MNorm, and DNorm of type integer. The transmission to the latter is done by reference and in this way more than one value is returned.

Input parameters:
int A
int M
int D
ref int ANorm
ref int MNorm
ref int DNorm

Return:
ref int ANorm ==> Number of years already normalized
ref int MNorm ==> Number of months already normalized
ref int DNorm ==> Number of days already normalized

Example:

```
using LN;
...
LN.LN OBJ = new LN.LN();
int AA = 0;
int MM = 0;
int DD = 0;
boolResult;
...
Result = OBJ.NormalizeDatesESP(0,135,105,ref AA, ref BB, ref DD);
...
```

Return:
In this example, it is considered that there are 0 years, 135 months and 105 days, which results in a normalized result of 11 years, with 6 months and 15 days.

true - If everything was OK

In this case it would be:
AA = 11
BB = 6
CC = 15

false - If there was an error

DATA VALIDATION METHODS

Usage: Winforms App & WEB App

public bool SSNCheck(string SSN)

DESCRIPTION:
This function validates the SSN input string that represents the Social Security Number of a person in the USA. The check understands only the syntax of the general format, but not the semantics. That is, it is only checked that three digits initially appear, followed by a hyphen, and finally four more digits.

General Format:
999-99-9999

INPUT PARAMETERS:

string SSN (Social Security)

RETURN:
true– If you meet the condition
false– If the condition is not met

public bool NUM(string Number)

DESCRIPTION:

This method checks whether a string is an unsigned integer, returning <true> if it is and

<false> if it is not.

INPUT PARAMETERS:
string Number

RETURN:
true – If you meet the condition
false – If the condition is not met

public bool NUMERIC(string Number)

DESCRIPTION:
This method checks if the input parameter is numeric in any of its integer or decimal representation variants (+11.23, -1.5, 4.2, -.123, +.6932, +321, -4, 56., -43., 2391, 001, etc.).

INPUT PARAMETERS:
string Number

RETURN:
true– If you meet the condition
false– If the condition is not met

public bool ZERO(string Number)

DESCRIPTION:
This method checks whether the input parameter contains zeros only, returning <true> if so.

INPUT PARAMETERS:
string Number

RETURN:
true- If Number contains only zeros.
false- Opposite case

public bool ALFANUM(string String)

DESCRIPTION:
This method checks if a string is alphanumeric, returning <true> if it is and <false> if it is not. This method considers a string to be alphanumeric when it contains only letters and numbers.

INPUT PARAMETERS:
string String

RETURN:
true- If the string is alphanumeric
false- If it is not

public bool ALF(string String)

DESCRIPTION:
This method checks if a string is alphabetic under the Spanish alphabet where accented letters, ñ, etc. exist, returning <true> if so and <false> if it is not.

A string is considered alphabetic in our alphabet when it contains only letters or the characters '-', or apostrophe (') appear in between and appear only once.

INPUT PARAMETERS:
string String

RETURN:
true- If the string is alphabetical
false- If it is not

Example:
Domínguez, Ramírez, Roberto O'Farril Sinforoso

public bool ALFEng(string String)

DESCRIPTION:
This method checks if a string is alphabetic under the English alphabet, but with the characteristic that accents, the letter ñ, or umlauts of the alphabet should not appear Spanish.

INPUT PARAMETERS:

string String

RETURN:
true- If the string is alphabetical
false- If it is not

Example:
Correct cases:
DOMINGUEZ, O'DONNEL, RAMIREZ, ROBERTO, O'FARRIL SINFOROSO

Incorrect Cases:
DOMÍNGUEZ, RAMÍREZ, ZUÑIGA

public bool Leap(string AAAAStr)

DESCRIPTION:
Function that determines if the entry year is a leap year, remember that in leap years February has 29 days and in the rest only 28.

INPUT PARAMETERS:
string AAAAStr

RETURN:
true– If you meet the condition
false– If the condition is not met

public bool BetweenInc(int N, int a, int b)

Description:
Method that determines whether the integer N is in the closed interval [a,b]
(includes the ends).

Input parameters:
int **N**
int **to**
int **b**

Return:
true- If it is in the range

false- If it is not in the range

public bool BetweenExc(int N, int a, int b)

Description:
Method that determines whether the integer N is in the open interval (a,b)
(excludes extremes).

Input parameters:
int **N**
int **to**
int **b**

Return:
true- If it is in the range
false- If it is not in the range

public bool BetweenIncDec(double N, double a, double b)

Description:
Method that determines whether the decimal number N is in the
closed interval [a,b] (includes the extremes).

Input parameters:
int **N**
int **to**
int **b**

Return:
true- If it is in the range
false- If it is not in the range

public bool BetweenExcDec(double N, double a, double b)

Description:
Method that determines whether the decimal number N is in the
open interval (a,b) (excludes extremes).

Input parameters:

intN
inta
intb

Return:
true- If it is in the range
false- If it is not in the range

bool VINCheck(string VIN)

Description:
This function validates a car's VIN input string representing its manufacturer ID with 17 chars.
Note:
Starting in 1980, it was internationally adopted to assign a unique identifier to each car, which has a formation law that contains the manufacturer's ID, and many other parameters. If you want to know more about this identifier, you can consult
Internet.

Input parameters:
string VIN (Car ID)

RETURN:
true– If the VIN is correct
false– If it is wrong

PROCESS EXECUTION METHODS

Usage: Winforms App & WEB App

public int MSDOSWait(string command)

Description:
This method executes an MS-DOS command in a console window, waiting for the execution of

the command to continue without closing said window waiting for the command to be executed.
terminate the execution of the invoked command.

Input parameters:
string command

RETURN:
Termination code of the invoked process (ExitCode)

public int MSDOSNoWait(string command)

Description:
This method executes an MS-DOS command in a console window, but does not remain in wait like the previous case. For this reason the task is executed asynchronously with respect to the calling program.

INPUT PARAMETERS:
string command

RETURN:
Termination code of the invoked process (ExitCode)

public void PAUSE()

DESCRIPTION:
Causes a pause in the MS-DOS command console (PAUSE) by issuing the following message:

***** PRESS ANY KEY TO CONTINUE *****

INPUT PARAMETERS:
Does not have

RETURN:
Does not have

public int ExecProcess(string command)

DESCRIPTION:
This method allows us to invoke any Windows object (any extension recognized by the system), therefore, if we invoke a .docx file and we have Office installed, then Windows opens the file with Word, and the same will happen with any other extension recognized by the system. With this method it is also possible to call any type of executable (.exe, .bat, .msi, msc, etc.).

Execution with this method is sequential, so that until the process with the invoked object finishes, it is not passed to the next C# instruction.

Below we show three examples of calls.

Example 1:

Open the document "ReporteMonsual.docx" found within the INFO folder of D:\

```
OBJLN.ExecProcess(@"D:\INFO\MonthlyReport.docx");
```

Example2:

(Running a .bat that is located in the root of D:\

```
OBJLN.ExecProcess(@"D:\ABC.bat");
```

Example3:

Execution of an .exe that appears inside a folder whose name contains spaces, in this case the PATH must be enclosed in quotes and following the C# syntax, for each quote that you want to insert in the specification string, you must put two, for example For this reason, three quotes appear at the beginning and at the end.

If you do not do this the referenced executable will not run
correctly.

```
OBJLN.ExecProcess(@"""C:\Renamer 2.20\ReNamer.exe""");
```

INPUT PARAMETERS:
string command

RETURN:
Termination code.

public int ExecProcessNoWait(string command)

DESCRIPTION:
This method is similar to ExecProcess but it is not sequential like the previous one that waits for the completion of the process to continue, here the invoked process is left running and the subsequent C# instructions of the program continue.

INPUT PARAMETERS:
string command

RETURN:
Termination code.

STRING FUNCTION METHODS

Usage: Winforms App & WEB App

Before beginning the description of each method in this section, it is worth clarifying that most are programmed using Regular Expressions. If the reader is not familiar with them, we refer them to the documentation of the Regex Test utility developed by the author and which is also available to anyone who wants it in the Windows Store.

public int FindStr(string String, string Substring)

DESCRIPTION:
This method calculates how many times <Substring> is contained within <String>.

Examples:
Ntimes = OBJ.FindStr(String, ":")

In this case, if string = "05:03:20" then Ntimes = 2, since the character ":" is contained twice within String.

INPUT PARAMETERS:
string String

string Substring

RETURN:
Int

public string FillString(string Char, int Ntimes)

DESCRIPTION:
Propagates the character <Car> (in quotes as a string) N times, returning a string with the result of the propagation.

INPUT PARAMETERS:
string **car**
int **Ntimes**

Return:
A string containing Car N times.

Example:

```
string CAD = "";
string CAR = "*";
int N = 5;
LN.LN OBJ = new LN.LN();
...
CAD = OBJ.FillString(CAR, N);
```

This leaves CAD = "*****";

public string DeleteChar(string String, string CHAR)

DESCRIPTION:
This function removes occurrences of the CHAR substring characters within **Chain**.

INPUT PARAMETERS:
string String
string CHAR
RETURN:string

Examples:
Example 1:
If Phone = "(786)905-1265" and we invoke the function as follows:
string Result = DeleteSTR(Phone, "()-");

This means that we want to delete the occurrences of the "()-" characters that are inside Phone and the result would be like this:
Result = "7869051265"

Note: It is convenient to clarify that in this method the search is carried out for each of the "()-" characters regardless of the order in which they appear, for this reason in the example above you could have specified this as a pattern: "-)(", and the result would be the same since what we are saying is that wherever you find some of those characters, delete them.

Example 2:

If CAD = "123:456:789" and we invoke the function as follows:

string Result = DeleteChar(CAD, ":");

This means that we want to delete the occurrences of the ":" character that is within CAD and the result would look like this:

Result = "123456789"

public string DeleteStr(string String, string PATTERN)

DESCRIPTION:
This function removes occurrences of the substring STR within String by returning the

resulting string.

INPUT PARAMETERS:
string String
string PATTERN
RETURN:
string

Example 1:
If CAD = "123abc4567abc890" and we invoke the function as follows:
string Result = DeleteSTR(CAD, "abc");
This means that we want to delete the "abc" substrings that are inside CAD and the result would be like this:

Result = "1234567890"

public void SplitString(ref string[ ] Result, string String, string[ ] delimiter)

DESCRIPTION:
Extracts the substrings separated by the delimiters specified in the third parameter of the string that is contained in String, leaving the substrings in the array of type String Result, where the transmission is by reference as shown below.
continuation.

Examples of Use:

Case 1:

In the sequence that appears below, the comma is taken as the only delimiter.

string[] RESULT = {""};

* The right part initializes the RESULT array, and this is required before using the SplitString function

string[] commas = new string[] {","}; -> Defines the separator as a comma

string source = "ONE, TWO, THREE, FOUR";

OBJ.SplitString(ref RESULT, source, commas);

In this case the following would remain in the RESULT array:
RESULT[0] = "ONE";
RESULT[1} = "TWO";
RESULT[2} = "THREE";
RESULT[3} = "FOUR";

Case 2:

In the sequence that appears below, several
delimiters, the comma, a period and a semicolon.

```
string[] delim = new string[] {",",".",";"};
string source = "ONE,TWO.THREE;FOUR";
OBJ.SplitString(ref RESULT, source, delim);
```

In this case the following would remain in the RESULT array:

```
RESULT[0] = "ONE";
RESULT[1] = "TWO";
RESULT[2] = "THREE";
RESULT[3] = "FOUR";
```

Input Parameters:

ref string[] Result,
ref delim String,
string[] delimiter

Return:
Does not have

GENERAL METHODS

Usage: Winforms App & WEB App

public double Percent(double P, double N)

DESCRIPTION:
This method returns the percentage P of the number N. For example, if we want to know 50%

of 20 we would call the function as follows:

double result = OBJ.Percent(50,20)

then result = 10

INPUT PARAMETERS:
doubleQ
doubleN

RETURN:
double

public string Trunc(double Origin, int Integers, int Decimals)

Description:
This function takes the double content of the Origin variable and represents it with the number of digits of the Integers variable for the integer part and the number of Decimals digits for the decimal part.

Return:
A string that contains the source double value and represents it in the number of digits that contain Integers and the number of digits that contain Decimals. The integer part is padded with leading zeros if necessary.

Example:

If Origin = 12.33444
Integers = 5
Decimals = 2

So:

LN.LN OBJ = new LN.LN();
...
OBJ.Trunc(12.33444, 5, 2)

returns the following string:
"00012.33"

public void Upper(ref string String)

DESCRIPTION:
Converts the input parameter string to uppercase. The transmission of parameters is by reference.
Example:
LN.LN OBJ = new LN.LN();
string CAD = "abc";
...
OBJ.Upper(CADref);
After executing the Upper method it would be:
CAD = "ABC"

Input Parameters:
string String
Return:
Does not have

public void Lower(ref string String)

DESCRIPTION:
Converts the input parameter string to lowercase. The transmission of parameters is by reference.

Example:
LN.LN OBJ = new LN.LN();
string CAD = "ABC";
...

OBJ.Lower(refCAD); // **ref**indicates that the transmission is by reference

After executing the Lower CAD method it would look like this:
CAD = "abc"

Input Parameters:
Chain

Return:
Does not have

public void ShowRPT(string PathReport)

Programmers: Geovany González and Héctor Fortún

Description:
The ShowRPT method is designed with the objective of quickly viewing a report static (.RPT) issued by the stand-alone Crystal Report utility. It is necessary to clarify that the reports generated dynamically by the program with the use of Crystal Report methods such as**SetParameterValue**A.M,**ExportToDisk,etc**, cannot be issued through ShowRPT.

This method is responsible for opening an autonomous window and displaying the content of the report using the Crystal Report Viewer, saving the programmer a large amount of programming and code time since ShowRPT takes care of everything.

EXAMPLE:
Assuming that we have previously generated the .RPT with the autonomous application of the Crystal Report in the following path:

"D:\REPORTS\MonthlyReport.rpt

Well, if we want our C# program to show the content of said report with the Crystal Viewer, it would be enough to put the following code:

```
LN.LN OBJ = new LN.LN(); // Instantiate a LN object
. . .
OBJ.ShowRPT(@"D:\REPORTS\MonthlyReport.rpt ");
```

*** THAT EASY ***

Input Parameters:
string PathReport

Return: Does not have

WORKING METHODS WITH DIRECTORIES

FileDiskOpenCreate method

FORMAT:

public StreamWriter FileDiskOpenCreate(string Path)

DESCRIPTION:

This method receives as input a string of characters that describes the directory where we are going to create the file described in Path.

If said directory contains sub-directories, these are not obtained by this method since for this purpose the method is available:

FoldersDiskOpenCreate
which is described later.

INPUT PARAMETERS:

string Path (contains the full path of the file to be created).

Note: We recommend that you use @"file_path" in order to use the normal backslash (\), without needing to duplicate them at each occurrence.

Example:

```
. . .
StreamWriterFileOut;
. . .

FileOut = FileDiskOpenCreate(@"D:\Employees\Department1\Dep1.txt");
```

RETURN:

1) Returns null if an error occurs when opening the file.

2) Returns an element of the typeStreamWriter that should be referenced by the method
FileDiskWrite
which is responsible for writing an article in the selected file and which describes later.

FileDiskOpenAppend method

FORMAT:

public StreamWriter FileDiskOpenAppend(string Path)

DESCRIPTION:

This method receives as input a string of characters that describes the directory that contains the file where we are going to add the new articles using the method
FileDiskWritewhich is described later.

If said directory contains sub-directories, these are not obtained by this method since for this purpose the method is available:

FoldersDiskOpenCreate
which is described later.

INPUT PARAMETERS:

string Path (contains the full path of the file to which new articles will be added).

Note: We recommend that you use @"file_path" in order to use the normal backslash (\), without needing to duplicate them at each occurrence.

Example:

```
. . .
StreamWriterFileOut;
. . .
FileOut = FileDiskOpenCreate(@"D:\Employees\Department1\Dep1.txt");
```

RETURN:

1) Returns null if an error occurs when opening the file.

2) Returns an element of the typeStreamWriter that should be referenced by the method
FileDiskWrite
which is responsible for writing/adding a new article to the selected file and
which is described later.

FileDiskOpenRead method

FORMAT:

public StreamReader FileDiskOpenRead(string Path)

DESCRIPTION:

This method receives as input a string of characters that describes the directory from which we are going to read the file described in Path.

INPUT PARAMETERS:

string Path (contains the full path of the file from which it will be read).

Note: We recommend that you use @"file_path" in order to use the normal backslash (\), without needing to duplicate them at each occurrence.

Example:

```
. . .
StreamReaderFileIn;
. . .

FileIn = FileDiskOpenRead(@"D:\Employees\Department1\Dep1.txt");
```

RETURN:

1) Returns null if an error occurs when opening the file.

2) Returns an element of the typeStreamReader that should be referenced by the method **FileDiskRead**
which is responsible for reading an article in the selected file and which is described later.

FileDiskWrite method

FORMAT:

public bool FileDiskWrite(StreamWriter OutFile,string Line)

DESCRIPTION:

This method writes the article contained in Line depending on how said file was opened (Create or Append).

INPUT PARAMETERS:

1)StreamWriter OutFile:

Here in OutFile the data type variable is recorded StreamWriter that was obtained in the return of one of the FileDiskOpenCreate or FileDiskOpenAppend methods.

2)string Line:

The Line variable contains the article in string format that you want to write to the open file

Example:

```
. . .
string Item = "";
StreamWriterFileOut;
. . .
FileOut= FileDiskOpenCreate(@"D:\Employees\Department1\Dep1.txt");
if(      FileOut==null)
         {
         ***Error handling sequence
         }
. . .
Article= "File=015,Name=Leslie Sullivan";
FileDiskWrite(FileOut,Article); // Write the previous article to the file Dep1.txt
. . .
. . .
```

RETURN:

1) Returns false if there is an error in writing the article.

2) Returns true if everything was OK.

FileDiskRead method

FORMAT:

public string FileDiskRead(StreamReader InFile)

DESCRIPTION:

This method reads the current article, returning it in string format.
This method is designed for .TXT files (it does not apply to binary files).

INPUT PARAMETERS:

1)StreamReader InFile:

Here in InFile the data type variable is recorded StreamReader which was obtained in the return FileDiskOpenRead method.

Example:

```
. . .
string Item = "";
StreamReaderFileIn;
. . .
FileIn= FileDiskOpenRead(@"D:\Employees\Department1\Dep1.txt");
if(      FileIn == null)
         {
         ***Error handling sequence
         }
. . .
// Receive the article in progress within the Dep1.txt file by depositing it in Article
Article = FileDiskRead(FileIn);
. . .
. . .
```

RETURN:
1) Returns null if there is an error in writing the article.

2) Return the current article if everything was OK.

FileDiskCloseWriter method

FORMAT:

public bool FileDiskCloseWriter(StreamWriter OutFile)

DESCRIPTION:

This method closes the file defined in**FileDiskOpenCreate**either **FileDiskOpenAppend**

INPUT PARAMETERS:

1)StreamWriter OutFile:

Here in OutFile the data type variable is recorded StreamWriter that was obtained in the return of one of the FileDiskOpenCreate or FileDiskOpenAppend methods.

Example:

```
. . .
string Item = "";
StreamWriterFileOut;
. . .
FileOut = FileDiskOpenCreate(@"D:\Employees\Department1\Dep1.txt");
if      (FileOut == null)
         {
         ***Error handling sequence
         }
. . .
Article = "File=015,Name=Leslie Sullivan";
FileDiskWrite(FileOut, Article); // Write the previous article to the file Dep1.txt
. . .
. . .
FileDiskCloseWriter(FileOut);
. . .
```

RETURN:

1) Returns false if there is an error in writing the article.

2) Returns true if everything was OK.

FileDiskCloseReader method

FORMAT:

public bool FileDiskCloseReader(StreamReader IntFile)

DESCRIPTION:

This method closes the file defined in**FileDiskOpenRead.**

INPUT PARAMETERS:

1)StreamReader InFile:

Here in InFile the data type variable is recorded StreamReader which was obtained in the return FileDiskOpenRead method.

Example:

```
. . .
string Item = "";
StreamReader FileIn;
. . .
FileIn = FileDiskOpenReader(@"D:\Employees\Department1\Dep1.txt");
if(      FileIn ==null)
          {
          ***Error handling sequence
          }
. . .
// Obtains the current article from the Dep1.txt file and deposits it inArticle
Article = FileDiskRead(FileIn);
. . .
. . .
FileDiskCloseReader(FileOut);
. . .
```

RETURN:

1) Returns false if there is an error in writing the article.

2) Returns true if everything was OK.

GetDirectoryFilesDialog method

FORMAT:

public string[] GetDirectoryFilesDialog(ref int NumberOfFiles)

DESCRIPTION:

This method returns the list of files selected within the dialog in an array of type string[] also leaving their number in the variable NumeroDeFicheros (transmission by reference so ref it is necessary).

INPUT PARAMETERS:

ref int NumberOfFiles

Variable transmitted by reference that obtains the number of files contained within the directory selected in the dialog. If the selected directory contains subfolders, these are not counted by this method because for this purpose the methods are used: **GetDirectoryFoldersDialog and GetDirectoryFoldersPath.**

Example:

```
. . .
int NumberOfFiles = 0;
. . .
string[] FileList = GetDirectoryFilesDialog(ref NumberOfFiles);
. . .
```

*Note:
In this example the list of files obtained is copied to the arrayFileList[] defined by the programmer, leaving their quantity in the integer variable of the second parameter

NumberOfFiles.

RETURN:

1) Returns null if an error occurs when opening the file.

2) Returns the list of files that belong to the selected directory in
FileList[]
and inthe second parameter (NumberOfFiles) the number of files.

GetDirectoryFilesPath method

FORMAT:

public string[] GetDirectoryFilesPath(string Path,ref int NumberOfFiles)

DESCRIPTION:

This method returns the list of files contained in the directory described in Path
in an array of type string[]and the number of them in the second variable
parameter FileNumber (transmission by reference so ref it is necessary).

INPUT PARAMETERS:

string Path
It is a string that defines the Path of the directory from which we want to obtain its list of files

ref int NumberOfFiles
Variable transmitted by reference that obtains the number of files in the chosen directory.

Example:

```
. . .
int NumberOfFiles = 0;
. . .
string[] FileList = GetDirectoryFilesPath(@"D:\Employees\Dep\Dep1.txt",
                                                    ref NumberOfFiles);
. . .
```

*Note:
In this example the list of files obtained is copied to the arrayFileList[] defined by the

programmer, leaving their quantity in the integer variable of the second parameter **NumberOfFiles**.

RETURN:

1) Returns null if an error occurs when opening the file.

2) Returns the list of files that belong to the selected directory and inthe second parameter (NumberOfFiles) the number of files in said list.

GetDirectoryFoldersDialog method

FORMAT:

public string[] GetDirectoryFoldersDialog(ref int NumberOfSubDirectories)

DESCRIPTION:
This method obtains the list of subfolders contained within the directory chosen by the user, returning the result in an array of type string[] and also depositing
the number of subfolders in the parameter variable NumeroDeSubDirectores
(transmission by reference so ref it is necessary).

INPUT PARAMETERS:

ref int NumberOfSubDirectories

Here is the number of subdirectories contained after the execution of the method
in the selected folder.

Example:

```
. . .
int NumberOfFolders = 0;
string[] ListSubFolders = null;
. . .
ListSubFolders = GetDirectoryFoldersDialog(ref NumberOfFolders);
. . .
```

*Note:
In this example the list of subfolders obtained is copied to the arrayListSubFolders[] defined

by the programmer, leaving the quantity of them in the integer variable of the parameter**NumberOfFolders**transmitted by reference.

RETURN:

1) Returns null if an error occurs when opening the file.

2) Returns the list of folders that belong to the selected directory, and inthe parameter NumberOfFolders the number of them.

GetDirectoryFoldersPath method

FORMAT:

```
public string[] GetDirectoryFoldersPath(string Path,ref int NumberOfSubDirectories)
```

DESCRIPTION:

This method obtains the list of subfolders contained within the directory chosen by the user in Path, returning the result in an array of type string[] and also depositingthe number of subfolders of said list in the variable of the second parameter
ref int NumberOfSubDirectories

(transmission by reference so ref it is necessary).

INPUT PARAMETERS:

string Path

Describes the Path of the directory that you want to list.

ref int NumberOfSubDirectories

Here is the number of subdirectories of the directory selected in Path

Example:

```
. . .
int NumberOfFolders = 0;
string[] ListSubFolders = null;
. . .
```

```
ListSubFolders = GetDirectoryFoldersPath(@"D:\Employees\Dep\Dep1.txt",
                                                    ref NumberOfFolders);
. . .
```

*Note:
In this example the list of subfolders obtained is copied to the arrayListSubFolders[] defined by the programmer, leaving their quantity in the integer variable of the second parameter**NumberOfFolders**.

RETURN:

1) Returns null if an error occurs when opening the file.

2) Returns the list of folders that belong to the selected directory and inthe second parameter is their quantity.

CreateFileParameter method

FORMAT:

```
public bool CreateFileParameter(string PathFile,string parameter)
```

DESCRIPTION:

This method writes the parameter line contained in Parameter into the file described in the PathFile that will be created.

INPUT PARAMETERS:

1)string PathFile
Contains the Path of the parameters file that will be created.

2)string parameters
Contains the parameter line that will be written to the given file.

Example:

```
. . .
CreateFileParameter(@"C:\Param.txt","ID=1020, Name=Rose, Age=25");
. . .
```

RETURN:

1) Returns false if an error occurs when opening the file.

2) Returns true if everything was OK.

CreateFileParameterArray method

FORMAT:

public bool CreateFileParameterArray(string PathFile,string[] Parameter)

DESCRIPTION:
This method writes the parameter lines contained in the Parameter array
of the second operand in the file described in PathFile that will be created.

INPUT PARAMETERS:

1)string PathFile
Describes the Path of the parameters file that will be created.

2)string[] Parameter

Array containing the parameter lines that will be written to the given file.

Example:

```
. . .
string[] Lines = {"Parameter1","Parameter2","Parameter3"};
CreateFileParameterArray(@"D:\Param.txt", Lines);
. . .
```

In this example the file is created **C:\Param.txt** and the articles are written:
Parameter1
Parameter2
Parameter3

RETURN:

1) Returns false if an error occurs when opening the file.

2) Returns true if everything was OK.
Note:
In Windows 10, the programmer must have writing rights enabled in the root of the C:\ drive if they want to create their parameter file in this place, since C:\ is usually the system drive and there is a higher level of protection on the part. of the system. For this reason we advise that you use any other disk for this purpose.

MethodStorageSize

FORMAT:

public string StorageSize(double Unit)

DESCRIPTION:

This method analyzes the Unit of storage expressed in bytes, and returns a substring expressed in the most convenient unit of storage equivalent to Unit.

INPUT PARAMETERS:

1)double Unit

Contains the size of the file expressed in bytes.

RETURN:

Unit >= 0 & Unit < 1000
returns Equivalent + "Bytes" --> Example "567 Bytes"

Unit >= 1000 & Unit < 1000000
returns Equivalent + " Kilobytes" --> Example "567 Kb"

Unit >= 1000000 & Unit < 1000000000
returns Equivalent + " Megabytes" --> Example "567 Mb"

Unit >= 1000000000 & Unit < 1000000000000)
returns Equivalent + "Gigabytes" --> Example "567 Gb"

Unit >= 1000000000000 & Unit < 1000000000000000
returns Equivalent + "Terabytes" --> Example "567 Tb"

Example:

```
. . .
doubleUnit = 800000;
stringResult = "";
Result =StorageSize(Unit);
* So in Result it would be: 800000/1000 = "800 Kb"
```